

CockroachDB

Day 1: Architecture & Foundations

 **Distributed SQL** — survive anything, scale anywhere

From Single Node to Multi-Region in Four Days

Course Overview — 4 Days

Day 1 — Architecture & Foundations

Distributed SQL, Raft & ranges, install & tooling, schema design (Labs 1–4)

Day 2 — Distributed Operations & Resilience

Transactions, query tuning, multi-region, schema patterns, throughput engineering (Labs 5–8)

Day 3 — Production Operations & Security

Observability & SLOs, capacity sizing, benchmarking, backup/DR, security hardening (Labs 9–12)

Day 4 — Advanced Features & Integration

CDC, outbox & idempotency, PostgreSQL migration, Kubernetes (Labs 13–16)

Hands-on: 16 labs, ~72% of class time. Everything runs in Docker — there is no `cockroach` binary to install.



Day 1 Schedule

Module 1: Architecture & Foundations

1. Introduction to CockroachDB

2. Core Architecture: Ranges, Raft, Leaseholders

3. Installation, Configuration & Tooling  Lab 1, Lab 2

4. SQL Dialect & Schema Design  Lab 3, Lab 4

1. Introduction to CockroachDB

What Is CockroachDB?

- Distributed SQL — a single logical SQL database spread across many nodes
- Strongly consistent by default (serializable isolation)
- Horizontally scalable — add nodes, get more capacity, no resharding
- PostgreSQL wire-compatible — drivers, ORMs, and most tools just work
- Survives disk, machine, rack, datacenter, and region failures

Built by ex-Googlers inspired by Google Spanner — but open source and self-hostable.

Why the Name?

Cockroaches are famous for surviving anything — nuclear winter, earthquakes, dropped boots.

CockroachDB is named for the same property: kill any node, any zone, any region, and the database keeps serving reads and writes.

This isn't a marketing claim — it's an architectural *invariant*. Every range of data is replicated to 3 (or 5, or 7) nodes by default, and writes only commit when a quorum of replicas has acknowledged them. Survivability is built into the storage layer, not bolted on as a failover script.

The Distributed SQL Landscape

System	Model	Wire Protocol	Notes
PostgreSQL	Single primary + read replicas	PG	Vertical scale; failover is operational
Google Spanner	Distributed SQL	gRPC / Spanner SQL	Managed-only; needs TrueTime atomic clocks
CockroachDB	Distributed SQL	PG	Self-hostable; HLC-based consistency
YugabyteDB	Distributed SQL	PG & Cassandra	Reuses PG query layer; different storage layer
TiDB	Distributed SQL	MySQL	HTAP-focused; TiKV + TiFlash

CockroachDB vs PostgreSQL

What's the Same

- SQL syntax (SELECT, JOIN, CTEs, window functions)
- PostgreSQL wire protocol
- Most drivers and ORMs
- Transactions, foreign keys, indexes

What's Different

- No stored procedures (functions only)
- Limited trigger support
- No extensions (no PostGIS, but spatial is built in)
- Always SERIALIZABLE — not READ COMMITTED by default
- Sequences are slower than UUID

When Is CockroachDB the Right Fit?

✓ Great Fit

- Always-on systems (no maintenance windows)
- Multi-region or global apps
- Outgrew a single PostgreSQL primary
- Regulated data with locality requirements (GDPR, data residency)
- Financial & transactional systems needing strong consistency

🤔 Probably Not

- Heavy OLAP / data-warehouse workloads (use a warehouse)
- Apps deeply dependent on PG extensions
- Very low concurrency, single-region, small data — PG is simpler
- Workloads dominated by long-running analytic queries

Real-World Use Cases

- Financial services — payments, ledgers, fraud (DoorDash, Bose, Lush)
- Retail & commerce — inventory, order management, carts
- Gaming — player state, leaderboards, in-game economies
- SaaS multi-tenant platforms — per-region tenant data residency
- Identity & authorization — globally-consistent session state

Common thread: *strong consistency + survivability + scale*. Any two are easy. All three is hard — and that's CockroachDB's niche.



2. Core Architecture

The Big Picture

From the top down, CockroachDB is a stack of layers — each one solves a specific problem:

Layers (top to bottom)

```
SQL          ← parser, planner, executor, optimizer
↓
Transaction  ← serializable isolation, 2-phase commit
↓
Distribution ← splits, merges, routing
↓
Replication  ← Raft consensus, leaseholders
↓
Storage (Pebble) ← LSM-tree, MVCC key-value store
```

Each node runs *all layers*. There is no separate "router" or "coordinator" tier.

It's All Key-Value Underneath

Every SQL row, every index entry, every system metadata record is stored as a key-value pair in Pebble (CockroachDB's LSM-tree storage engine).

SQL → KV Example

```
-- Logical SQL:
CREATE TABLE users (id UUID PRIMARY KEY, email STRING, name STRING);
INSERT INTO users VALUES ('abc-123', 'jane@example.com', 'Jane');

-- Physical KV (simplified):
/Table/users/primary/'abc-123'/email → 'jane@example.com'
/Table/users/primary/'abc-123'/name  → 'Jane'
```

Why this matters: the layout of your primary key directly determines how rows are distributed across the cluster. Sequential keys = hotspots. Random keys = balanced load. We'll dig into this in Module 1 Section 4.

Ranges — The Unit of Distribution

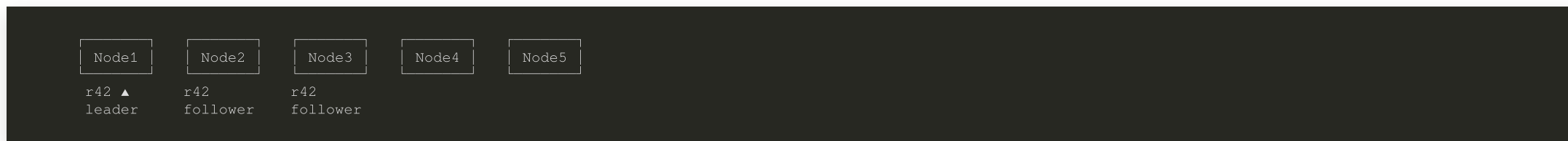
- The KV keyspace is sorted and divided into contiguous ranges
- Default range target size: ~512 MiB
- When a range grows past the target, it splits automatically
- When ranges shrink, they can merge
- Each range is independently replicated and routed

A table doesn't live "on a node" — it lives across however many ranges its data fills, and each range can be on a different set of nodes.

Replication — 3 Copies by Default

Every range is replicated to N nodes (default $N=3$). The replicas form a Raft group.

Range r42 spread across a 5-node cluster



The 3 replicas of r42 elect a leader via Raft. The leader proposes writes; followers ack. A write commits when a majority (2 of 3) acknowledges — so the cluster tolerates losing 1 replica per range with zero data loss and no manual failover.

Raft Consensus in 60 Seconds



What Raft guarantees

- All replicas agree on the *order* of writes
- A committed write is durable as long as a majority of replicas survive
- If the leader dies, followers elect a new one



Quorum math

- 3 replicas → tolerate 1 loss
- 5 replicas → tolerate 2 losses
- 7 replicas → tolerate 3 losses

Odd numbers because majority = $\lfloor N/2 \rfloor + 1$.

Leaseholder — Single Source of Truth for Reads

One of the replicas in each Raft group is also designated the leaseholder. The leaseholder serves all reads for its range without needing to talk to the other replicas.

- Leader (Raft): proposes writes; usually same node as leaseholder
- Leaseholder: serves reads; coordinates writes
- Followers: apply Raft log; can serve *follower reads* (stale snapshots)

Reads from a leaseholder are strongly consistent — they reflect every committed write. The leaseholder's lease is time-bounded (~9s), so a dead leaseholder is replaced quickly by a peer.

Multi-Active Availability

Traditional Primary/Replica

- One writable primary at a time
- Failover takes seconds to minutes
- Risk of split-brain & data loss
- Replicas are read-only

CockroachDB Multi-Active

- Every node accepts reads & writes
- No failover — quorum carries on
- No split-brain (Raft prevents it)
- No "follower" lag for writes

Apps point at any node — typically through a load balancer — and the cluster routes the request to the right leaseholder internally.

Node Communication & Gossip

Every node maintains a cluster-wide view of *metadata* via a gossip protocol:

- Liveness — which nodes are alive (heartbeats)
- Range descriptors — where each range's replicas live
- Capacity & load — used by the rebalancer
- Node attributes & locality — region, zone, rack

Gossip is *eventually consistent* — but the actual data and Raft log are *strongly consistent*. The distinction matters: gossip just tells nodes where to look; consistency comes from Raft.

Cluster Membership & Join Flow

Bringing a new node online

1. Operator runs: `cockroach start --join=<existing-node>:26257`
2. New node connects to one or more existing nodes (the "join list")
3. Gossip propagates the new node's address & locality
4. The allocator decides which ranges should add this node as a replica
5. Raft snapshots stream existing data to the new node
6. New node is now a full participant — no manual intervention required

Lab 1 does both: `scripts/crdb add-node`, then `scripts/crdb run node decommission 3 --insecure`. Decommissioning is *refused* when the replicas have nowhere to go — as on a 3-node cluster at $RF=3$, which is why the lab adds a fourth node first.

Architecture Quick Quiz

Q1. Range r42 has 3 replicas. Two nodes hosting those replicas go down at the same time. What happens?

A: The remaining replica cannot achieve quorum, so r42 becomes unavailable for reads and writes until a peer comes back online. Other ranges with surviving quorum keep serving.

Q2. An application connects to Node 3 and runs `SELECT * FROM users WHERE id = 'abc'`. The leaseholder for that row is on Node 1. What does Node 3 do?

A: Node 3 routes the request to Node 1's leaseholder via internal RPC and returns the result. The client never sees the redirect.

3. Installation, Configuration & Tooling

One Binary, No Dependencies

CockroachDB ships as a single statically-linked binary called `cockroach`. It contains:

- The server (every node, every role)
- The SQL client (`cockroach sql`)
- The demo cluster runner (`cockroach demo`)
- Backup/restore, debug, decommission, and admin tooling

No JVM. No Python runtime. No package manager dependencies. `scp` the binary, run it, done.

Installing

macOS (Homebrew)

```
brew install \
  cockroachdb/tap/cockroach

cockroach version
```

Linux / macOS (manual)

```
curl https://binaries.cockroachdb.com/\
cockroach-latest.linux-amd64.tgz | tar -xz

sudo mv cockroach-*/cockroach \
  /usr/local/bin/

cockroach version
```

Windows: download the ZIP from the [releases page](#), extract, and add the folder to PATH.

Three Ways to Run a Cluster

Command	What you get	Use for
<code>cockroach demo</code>	In-memory multi-node cluster	Learning, demos, this course
<code>cockroach start-single-node</code>	One persistent node, no replication	Dev workstation
<code>cockroach start</code>	Real cluster node, joins via <code>--join</code>	Staging & production

cockroach demo — A Cluster With No Setup

Everything runs in a container — there is no binary to install

```
# 3-node empty cluster, entirely in memory
docker run --rm -it cockroachdb/cockroach:v23.2.5 \
  demo --nodes 3 --no-example-database --empty

# 9-node, 3-region cluster with simulated WAN latencies - this is Lab 7
docker run --rm -it -p 8090:8080 cockroachdb/cockroach:v23.2.5 \
  demo --global --nodes 9 --no-example-database --empty --http-port=8080
```

demo is for *throwaway* clusters. Every other lab uses the persistent compose cluster. Lab 7 needs demo: `--global` simulates inter-region latency, and it ships the licence multi-region SQL requires.

- Drops you straight into a SQL shell connected to the first node
- DB Console URL is printed at startup
- Everything is in memory — `Ctrl+D` or `\q` destroys the cluster
- Survives kills inside the demo (`cockroach node decommission`, etc.)

cockroach start — How a Cluster Forms

You type one line...

```
scripts/crdb up
```

...and `docker/labs.yml` runs this, once per node

```
cockroach start --insecure \  
  --advertise-addr=crdb1 \  
  --http-addr=0.0.0.0:8080 \  
  --join=crdb1:26257,crdb2:26257,crdb3:26257  
  
# Every node gets the SAME --join list. None of them is the "master".  
# Then, exactly once in the life of the cluster:  
cockroach init --insecure --host=crdb1:26257
```

Read it in `docker/labs.yml`; watch a node join with `scripts/crdb logs 1`. **--insecure** is for labs only — Lab 12 builds a TLS cluster.

The SQL Shell

Connecting to a running cluster

```
# Built-in client, run inside node 1 — the command you will use all week
scripts/crdb sql
scripts/crdb sql -e "SELECT count(*) FROM lab1.notes;"

# Any PostgreSQL client, over the published port (psql, pgcli, DBeaver, ...)
psql "postgresql://root@localhost:26257/defaultdb?sslmode=disable"
```

Inside the shell, common helpers:

- `SHOW DATABASES;` — list databases
- `SHOW TABLES;` — list tables in current db
- `SHOW CREATE TABLE foo;` — show DDL
- `\?` — list shell commands
- `\q` — quit

Two Addresses, One Cluster

From your machine the cluster is `localhost:26257`.

From inside another container it is `crdb1:26257`.

Mixing these up is the single most common connection error in this course.

It bites hardest where the *database* dials out — a changefeed sink in Lab 13, a workload generator in Lab 8. Those run inside the cluster network, so `localhost` there means the node itself, not your laptop.

DB Console — The Web UI

Every node exposes a built-in web console on port 8080 (configurable). It is read-mostly: metrics, jobs, statements, schema, and cluster topology.

Where to find it

```
scripts/crdb console → http://localhost:8080 (node 1)
                     http://localhost:8081 (node 2)
                     http://localhost:8082 (node 3)

Every node serves the same console. If one is down, read another.
```

Key tabs we'll tour in Lab 2:

- Overview — nodes, ranges, capacity at a glance
- Metrics — built-in dashboards (Overview, Hardware, Runtime, SQL, Replication, Storage)
- SQL Activity — top statements, recent statements, sessions, transactions
- Jobs — long-running things like BACKUP, RESTORE, CHANGEFEED, schema changes
- Databases — schema browser with per-table stats

PostgreSQL Wire-Protocol Compatibility

CockroachDB speaks the PostgreSQL wire protocol, so most PG-aware tooling works out of the box:

Works

- psql, pgcli, DBeaver, JetBrains DataGrip
- JDBC, psycopg, pgx, node-postgres, asyncpg
- SQLAlchemy, GORM, Prisma, Hibernate (with CRDB dialect)
- Most BI tools (Metabase, Superset, Tableau)

Caveats

- No stored procedures (functions only)
- No PG extensions (PostGIS-style features are built in)
- Some system catalog rows differ — tools doing deep introspection may need a CRDB-aware adapter
- SERIAL behaves differently — see Section 4



Lab 1: Cluster Bootstrap & Lifecycle



75 min

Objectives

- Spin up a 3-node demo cluster and confirm it is healthy
- Inspect node liveness, ranges, and replica placement
- Kill a node and observe Raft's automatic recovery
- Connect with `scripts/crdb sql`, `psql`, and a Python driver



Quick Start

```
scripts/crdb up # 3-node cluster in Docker
```

When you see the `defaultdb>` prompt, note the DB Console URL printed above it.

Lab Activities

- Part A: Start the cluster, list nodes, find leaseholders
- Part B: Kill node 2, watch replication self-heal
- Part C: Connect with `psql` and verify wire compatibility

Reading the DB Console — Cluster Health

After Lab 1, you've seen the console. Here's what to look at *first* when something feels wrong:

- Overview → Node list: are all nodes *live*? Any in "suspect" or "decommissioning"?
- Overview → Capacity: storage usage per node — imbalance is a red flag
- Metrics → Replication: under-replicated ranges > 0 means data is at risk
- Metrics → SQL: connection count, statement latency, error rate
- Hot Ranges: top ranges by QPS — your hotspot suspects live here

System Tables You'll Use

Useful built-in views

```
-- All nodes in the cluster
SELECT node_id, address, locality, is_live FROM crdb_internal.gossip_nodes;

-- All ranges and where their leaseholders live.
-- (crdb_internal.ranges_no_leases has no table_name or lease_holder columns -
-- anything table-scoped or lease-related comes from SHOW RANGES.)
SELECT table_name, range_id, lease_holder, range_size_mb
FROM [SHOW CLUSTER RANGES WITH TABLES, DETAILS] LIMIT 20;

-- Top statements by execution count
-- (there is no `key` column - the statement text lives in metadata->>'query')
SELECT metadata->>'query' AS statement,
       (statistics->'statistics'->>'cnt')::INT AS executions,
       (statistics->'statistics'->'svcLat'->>'mean')::FLOAT AS mean_latency_s
FROM crdb_internal.statement_statistics
ORDER BY executions DESC LIMIT 10;

-- Current sessions and what they're doing
SELECT node_id, session_id, application_name, active_queries
FROM crdb_internal.cluster_sessions;
```



Lab 2: DB Console & SQL Operational Tour



70 min

Objectives

- Tour every major DB Console tab against live traffic
- Generate load with the built-in `workload` tool
- Read `crdb_internal` tables to answer operational questions



Setup

```
# Lab 1 cluster still running? Good. Otherwise:  
scripts/crdb up          # 3-node cluster in Docker
```

Lab Activities

- Part A: Tour Overview, Metrics, Databases, Jobs tabs
- Part B: Run `scripts/crdb run workload run kv` to generate load and watch the dashboards react
- Part C: Query `crdb_internal` to find the hottest ranges and slowest statements

4. SQL Dialect & Schema Design

The Idea This Course Is Built Around

In a single-node database, the primary key answers "*which row is this?*"

In a distributed database, it also answers "which machine does this row live on, and which machine gets the write?"

The primary key is a distribution decision. Uniqueness is the easy half.

Day	What that decision turns into
1	Range layout and where the hotspot is
2	Contention, plan shape, and measured rows/sec (Lab 8)
3	Write amplification → node count → hardware bill (Lab 10)
4	Outbox design, migration redesign, Kubernetes topology

You will be asked to defend a primary key with a number before this course is over.

The Most Important Decision: Primary Keys

The primary key is the physical sort order of a table's KV data. Choose it badly and you create a hotspot; choose it well and the cluster spreads load evenly.

Classic Anti-Pattern

```
CREATE TABLE events (  
  id      SERIAL PRIMARY KEY,  
  payload JSONB,  
  created TIMESTAMPTZ DEFAULT now()  
);
```

Sequential `id` values → every insert lands on the rightmost range → one leaseholder is hammered while others idle.

SERIAL Is Not What You Think

In PostgreSQL, `SERIAL` is a sequence-backed counter (1, 2, 3, ...). In CockroachDB, `SERIAL` is not sequential — by default it's `unique_rowid()`:

`unique_rowid()` — 64-bit integers, time-prefixed

| 48 bits HLC timestamp | 16 bits node ID + counter |

Better than PG's `SERIAL` — values are roughly time-ordered but each node carves out its own counter space, so writes spread across the cluster.

Still not random — values are *monotonic*, so all inserts cluster on the latest range. For high-throughput inserts, prefer `UUID`.

UUID — The Default Choice

Recommended pattern for high-write tables

```
CREATE TABLE events (  
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  payload JSONB,  
  created TIMESTAMPTZ DEFAULT now()  
);
```

- `gen_random_uuid()` produces 128 random bits — writes scatter across all ranges
- No "rightmost range" hotspot
- Tradeoff: bigger keys (16 bytes vs 8) and no natural ordering

Watch out: if you need *ordered* iteration by insert time, add a `created_at` index — don't try to encode ordering into the primary key.

Sequences — Use Sparingly

When you really need gapless integers

```
CREATE SEQUENCE invoice_seq;  
  
CREATE TABLE invoices (  
  id INT8 PRIMARY KEY DEFAULT nextval('invoice_seq'),  
  ...  
);
```

Performance trap: every `nextval()` is a distributed transaction — it must atomically increment a shared counter range. High-volume inserts on a sequence-PK table will *not* scale.

Use sequences when business logic *requires* monotonic integers (e.g. invoice numbers for accounting). Otherwise, use UUID.

Hash-Sharded Indexes — Spread Sequential Keys

When you genuinely need a monotonic key, hash sharding spreads it across N buckets:

Hash-sharded primary key

```
CREATE TABLE events (  
  account_id UUID,  
  created    TIMESTAMPTZ DEFAULT now(),  
  id         UUID DEFAULT gen_random_uuid(), -- per-row tiebreaker  
  payload    JSONB,  
  PRIMARY KEY (account_id, created, id) USING HASH WITH (bucket_count = 16)  
);
```

Why **id** is in the key: `now()` is the *transaction* timestamp, so every row written by one statement shares it. `(account_id, created)` alone is not unique and the second row fails with duplicate key value violates unique constraint. A time-series key needs a per-row tiebreaker.

Under the hood a hidden `..._shard_16` column (bucket 0–15) is prepended to the key, so near-identical timestamps land on different ranges. Point lookups by `(account_id, created)` still work; open-ended scans on `created` alone may want a separate non-sharded index.

Composite Primary Keys

Common multi-tenant pattern

```
CREATE TABLE orders (  
  tenant_id UUID,  
  order_id  UUID DEFAULT gen_random_uuid(),  
  customer  STRING,  
  total     DECIMAL(12,2),  
  PRIMARY KEY (tenant_id, order_id)  
);
```

- (tenant_id, order_id) co-locates one tenant's data in a contiguous key range
- Queries filtering by tenant become single-range scans (fast)
- Sets up cleanly for geo-partitioning later (Day 2 — REGIONAL BY ROW)
- Hot tenants can still cause a hotspot — be aware

Data Types — PG-Compatible, with Care

Use freely

- UUID
- INT8 (always 64-bit)
- STRING (variable)
- BYTES
- TIMESTAMPTZ (TZ-aware)
- DECIMAL (exact)
- JSONB
- BOOL

Watch out

- INT = INT8 (PG defaults to INT4!)
- FLOAT = FLOAT8
- SERIAL $\neq$ PostgreSQL SERIAL
- TIMESTAMP is TZ-naive — prefer TIMESTAMPTZ
- NUMERIC = DECIMAL

Constraints & Foreign Keys

Standard SQL, with one wrinkle

```
CREATE TABLE customers (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email STRING UNIQUE NOT NULL CHECK (length(email) < 256)  
);  
  
CREATE TABLE orders (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  customer_id UUID NOT NULL REFERENCES customers(id) ON DELETE CASCADE,  
  total DECIMAL(12,2) NOT NULL CHECK (total >= 0)  
);
```

Foreign keys are real and enforced — the *parent* column needs a unique index, which the primary key provides. Cascading actions work as in PG.

The child column gets no index. `orders.customer_id` above is unindexed until you say `CREATE INDEX ON orders(customer_id)`. Until then every join or lookup by customer scans the whole `orders` table — the most common slow join in practice.

Indexes — Same Idea, Slightly Different Costs

- Secondary indexes are themselves ranges — they replicate just like data
- Each write to an indexed table writes to *every* covering index
- Indexes can be partial (`WHERE` predicate), covering (`STORING` clause), or hash-sharded
- CockroachDB collects table statistics automatically; the cost-based optimizer uses them

Common patterns

```
-- Plain index
CREATE INDEX ON orders(customer_id);

-- Covering / "storing" index — extra columns travel with the key
CREATE INDEX ON orders(customer_id) STORING (total, status);

-- Partial index — only index "open" orders
CREATE INDEX ON orders(customer_id) WHERE status = 'open';

-- Hash-sharded index — avoid hotspots on time-series patterns
CREATE INDEX ON events(account_id, created)
  USING HASH WITH (bucket_count = 8);
```



Schema Patterns Playbook

A take-home catalog for high-volume workloads, exercised in Labs 3–5. Full schemas:

`labs/SCHEMA_PATTERNS_PLAYBOOK.md`.

● Patterns to reach for

- #1 Hash-Sharded Time-Series PK — writes don't pile on the right edge
- #2 Append-Only Event Log — UUID PK + TTL
- #3 Per-Tenant Co-located PK — single-range scans per tenant
- #4 Time-Bucketed Composite PK — fast time-range queries
- #5 Sharded Counter — no more 40001 retry storms
- #6 Hot/Cold Split — partial index over hot rows
- #7 GLOBAL Reference Table — read-anywhere lookup
- #8 Outbox + CDC — atomic publish
- #9 TTL Table — automatic expiry
- #10 Pre-Split + Bulk Import

● Anti-patterns to catch in review

- SERIAL on hot paths
- Single-row UPDATE counter SET $n = n + 1$
- Low-cardinality lead column in PK
- Indexes on every column
- DELETE-as-you-go cleanup
- App writes to DB *and* Kafka
- LOCALITY GLOBAL on a write-heavy table

Patterns You Already Know

Several patterns map directly onto things we've covered in this section:

Pattern	What you've seen	Lab
#1 Hash-Sharded Time-Series PK	<code>USING HASH WITH (bucket_count = ...)</code>	Lab 3 Part D
#3 Per-Tenant Co-located PK	<code>PRIMARY KEY (tenant_id, ...)</code>	Lab 3 Part C
#6 Hot/Cold Split	<code>CREATE INDEX ... WHERE status = 'open'</code>	Lab 4 Part C
#10 Pre-Split + Bulk Import	<code>ALTER TABLE ... SPLIT AT VALUES (...)</code>	Lab 3 Part E

Lab 3 also builds #2 (Append-Only Event Log), #8 (Outbox + CDC), and #9 (TTL Table) from scratch.



Lab 3: Schema Design — High-Volume Patterns & Hotspot Avoidance

90 min

Objectives

- Observe a hotspot live with `SERIAL`; fix with UUID, composite, hash-sharded PKs
- Pre-split a table and bulk-load without rightmost-range pile-up
- Build an Append-Only Event Log with TTL for automatic expiry
- Build the Outbox Pattern and verify atomic business-write + event publish
- Apply a code-review checklist to 5 PK shapes



Setup

```
scripts/crdb-up # 3-node cluster in Docker
```

Lab Activities

- Part A: Create `events_serial`; observe hotspot
- Part B: Re-create with `UUID` PK; compare distribution
- Part C: Composite PK `(tenant, id)` for co-location
- Part D: Hash-sharded composite for a hot tenant
- Part E: Pre-splits + bulk insert
- Part F: PK code-review checklist
- Part G: Append-Only Event Log + TTL job
- Part H: Outbox Pattern + Core changefeed

Index Picking — A Mental Model

The optimizer picks an index by estimating how many rows it will read. The fastest plan is the one that *scans the fewest keys*.

Reading a plan

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 'abc';

• scan
  table: orders@orders_customer_id_idx      ← used the secondary index
  spans: ['/abc' - /'abc']                  ← single-key span = great
```

Anti-pattern signals: full table scan on a large table, span of [-] (the whole keyspace), or an estimated row count near the table cardinality.

Storing Clause — "Index-Only" Reads

If a query reads columns not in the indexed key, CockroachDB does an extra primary-key lookup per row. `STORING` avoids that:

Without vs with `STORING`

```
-- Without STORING: index lookup, then PK lookup per row
CREATE INDEX ON orders(customer_id);
SELECT customer_id, total FROM orders WHERE customer_id = 'abc';

-- With STORING: index alone has everything we need
CREATE INDEX ON orders(customer_id) STORING (total, status);
SELECT customer_id, total, status FROM orders WHERE customer_id = 'abc';
```

Cost: every `UPDATE` to a stored column must also update the index. Don't blindly add every column to `STORING` — pick the ones that benefit the hot read path.

Anti-Patterns Cheat-Sheet

Pattern	Why It Hurts	Fix
Sequential PK (<code>SERIAL</code> , sequence)	Rightmost-range hotspot	UUID or hash-sharded composite
(<code>timestamp</code>) as PK / index lead	Same — all writes on latest range	Hash-sharded index
Big tables, no FK index	FK checks become full scans	Index every FK column
<code>SELECT *</code> everywhere	Forces PK lookup, defeats covering indexes	List columns; add <code>STORING</code>
Implicit transactions for chained statements	Lock contention, retries	Explicit <code>BEGIN ... COMMIT</code>



Lab 4: Indexing Strategies — Every Index Type That Matters



70 min

Objectives

- Build secondary, partial, covering, and hash-sharded indexes
- Use `EXPLAIN` to confirm which index a query uses
- Quantify the cost of an unused index on writes
- Apply `STORING` to eliminate a per-row PK lookup



Setup

```
docker-compose up # 3-node cluster in Docker
```

Lab Activities

- Part A: Create the orders schema; observe full-scan plans without indexes
- Part B: Add secondary, partial, and `STORING` indexes; verify plans
- Part C: Build a hash-sharded time-series index and run a range-by-time query

Day 1 Wrap-Up

What you can now do

- Explain ranges, Raft, leaseholders, and multi-active availability
- Start, connect to, and inspect a CockroachDB cluster
- Read the DB Console for cluster health and hot ranges
- Choose primary keys that *don't* create hotspots
- Apply hash sharding, STORING, and partial indexes appropriately

Day 2 preview

- Transactions, contention, retries, and EXPLAIN ANALYZE
- Multi-region topologies and survival goals
- Backup, restore, changefeeds, security, and monitoring